Ενδεικτικές Λύσεις Πρώτης Εργασίας

Λύση Πρώτου Ζητήματος

Πριν προχωρήσουμε σε οποιαδήποτε ενέργεια εκτελούμε το επερώτημα και καταγράφουμε στατιστικά στοιχεία σχετικά με την εκτέλεσή του.

STATISTICS IO

```
(5 rows affected)
Table 'Users'. Scan count 1, logical reads 5716, physical reads 2, read-ahead
reads 5717, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.
```

STATISTICS TIME

```
SQL Server Execution Times:
   CPU time = 78 ms,  elapsed time = 166 ms.
```

1. Επιλέγουμε να δημιουργήσουμε το ευρετήριο με την μεγαλύτερη επιλεξιμότητα (selectivity):

```
Select CAST (count(distinct UserLocation) as float) / count(*)
       as LocationAverageSelectivity
FROM users
```

LocationAverageSelectivity
0,0409623342919268

```
Select CAST (count(distinct CreationDate) as float) / count(*)
       as CreationDateAverageSelectivity
 FROM users
```

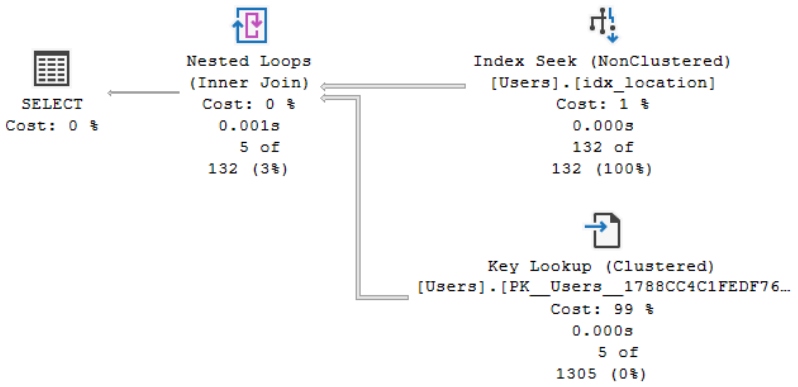
CreationDateAverageSelectivity
0,00295260139547156

Βάσει των παραπάνω επιλέγουμε το ευρετήριο:

```
create index idx_location ON users(UserLocation)
```

2. Δημιουργούμε το ευρετήριο και εκτελούμε το επερώτημα. Στο πλάνο εκτέλεσης του επερωτήματος βλέπουμε ότι χρησιμοποιείται το ευρετήριο που επιλέξαμε.

```
Query 1: Query cost (relative to the batch): 100%
SELECT [DisplayName] FROM [users] WHERE [CreationDate]=@1 AND [Userlocation]=@2
```



STATITICS IO

```
(5 rows affected)
Table 'Users'. Scan count 1, logical reads 751, physical reads 2, read-ahead
reads 994, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.
```

STATISTICS TIME

```
 SQL Server Execution Times:
   CPU time = 31 ms,  elapsed time = 57 ms.
```

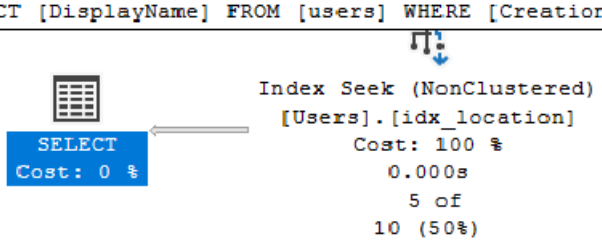
3. Παρατηρώντας το πλάνο εκτέλεσης του επερωτήματος διαπιστώνουμε ότι το μεγαλύτερο κόστος έχει ο τελεστής **Key Lookup** (Key Lookup operator) το οποίο μπορούμε να απαλείψουμε αν στο ευρετήριο συμπεριλάβουμε τα πεδία **creationDate** και **DisplayName.**

Δημιουργούμε το παρακάτω ευρετήριο:

```
create index idx_location ON users(UserLocation)
                          INCLUDE (creationDate, DisplayName)
```

Ακολουθεί το νέο πλάνο εκτέλεσης καθώς και στατιστικά στοιχεία που αποδεικνύουν ότι το προτεινόμενο ευρετήριο βελτιστοποιεί την εκτέλεση του **E1**.

```
Query 1: Query cost (relative to the batch): 100%
SELECT [DisplayName] FROM [users] WHERE [CreationDate]=@1 AND [Userlocation]=@2
```



STATISTICS IO

```
(5 rows affected)
```

```
Table 'Users'. Scan count 1, logical reads 5, physical reads 1, read-ahead
reads 2, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.
```

STATISTICS TIME

```
SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 15 ms.
```

Λύση Δεύτερου Ζητήματος

1.

Βήμα Πρώτο

Αρχικά εκτελούμε το επερώτημα δίχως την δημιουργία κάποιου ευρετηρίου: Ακολουθούν τα στατιστικά στοιχεία από την εκτέλεση του επερωτήματος:

STATISTICS IO

(283 rows affected)

Table 'Tags'. Scan count 1, logical reads 118, physical reads 1, read-ahead reads 123, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'Posts'. Scan count 0, logical reads 2141, physical reads 1, read-ahead reads 3312, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'PostTags'. Scan count 5, logical reads 6671, physical reads 1, read-ahead reads 6621, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

STATISTICS TIME

(283 rows affected)

SQL Server Execution Times:

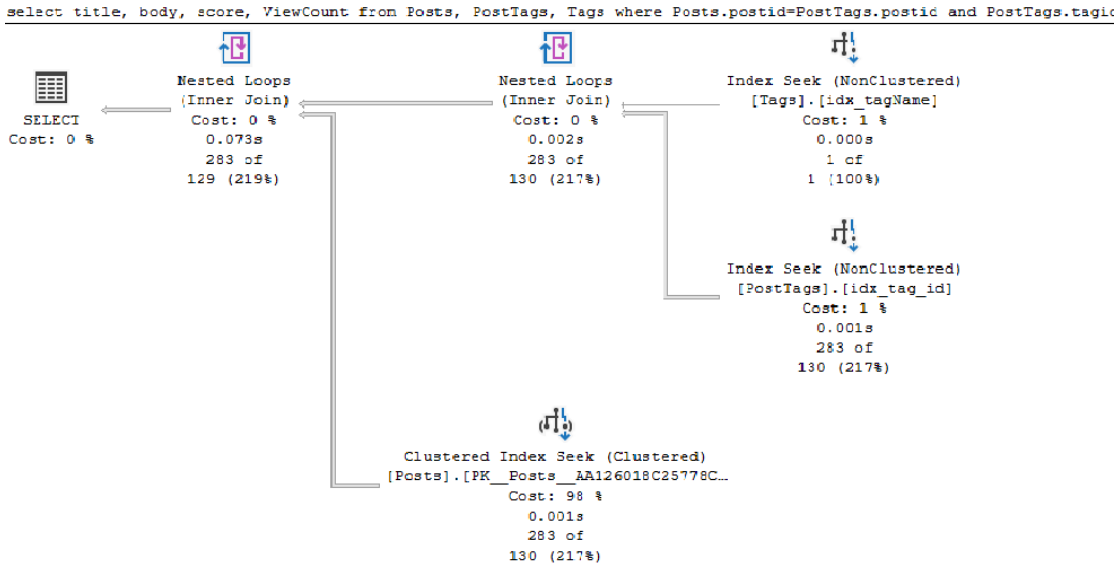
CPU time = 155 ms, elapsed time = 315 ms.

Βήμα Δεύτερο

Στην συνέχεια δημιουργούμε τα ακόλουθα ευρετήρια:

```
create unique index idx_tagName ON Tags(TagName)
create index idx_tag_id ON PostTags(tagId)
```

Ακολουθεί το πλάνο εκτέλεσης του επερωτήματος και τα αντίστοιχα στατιστικά στοιχεία:



STATISTICS IO

(283 rows affected)

Table 'Posts'. Scan count 0, logical reads 2134, physical reads 1, read-ahead reads 3312, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'PostTags'. Scan count 1, logical reads 3, physical reads 3, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

Table 'Tags'. Scan count 0, logical reads 2, physical reads 2, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.

TIME IO

(283 rows affected)

SQL Server Execution Times:

CPU time = 16 ms, elapsed time = 213 ms.

Βήμα Τρίτο

Βλέπουμε στο πλάνο εκτέλεσης ότι ο SQL Server χρησιμοποιεί το ευρετήριο **idx_tagName** για να εντοπίσει τον όρο Informix και να ανακτήσει το κωδικό του **(tagId).** Στην συνέχεια, χρησιμοποιεί (index seek) το δεύτερο ευρετήριο που δημιουργήσαμε **(idx_tag_id)** για να εντοπίσει τους κωδικούς των σχετικών αναρτήσεων **(PostId)** από τον πίνακα PostTags.

Μπορούμε να πετύχουμε το ίδιο δίχως την δημιουργία του ευρετηρίου **idx_tag_id** αν αλλάξουμε την μορφή του πρωτεύοντος κλειδιού του πίνακα PostTags από **PRIMARY KEY (PostId, TagId)** σε **PRIMARY KEY (TagId, PostId)**

Κάνουμε την αλλαγή εκτελώντας τις παρακάτω εντολές:

```
ALTER TABLE PostTags
   DROP CONSTRAINT PK__PostTags__7C45AF82E9C6EE29

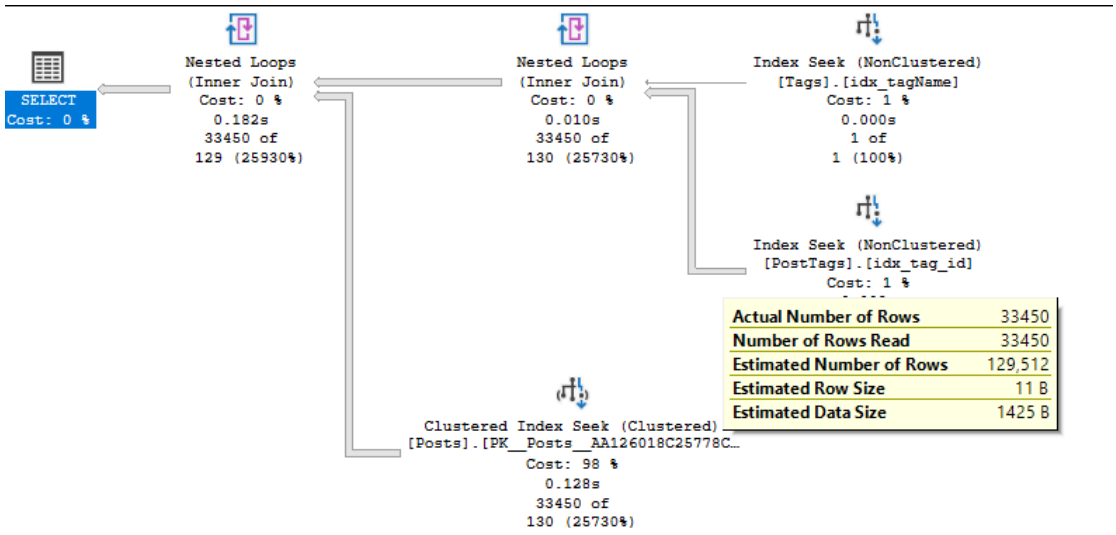
ALTER TABLE PostTags
   ADD CONSTRAINT pk_idx_tagid_postid PRIMARY KEY CLUSTERED (tagid,postid)
```

Εκτελούμε εκ νέου το επερώτημα E2 και διαπιστώνουμε ότι το πλάνο εκτέλεσης είναι ίδιο με το προηγούμενο (βλέπε βήμα 2).

Με τον τρόπο αυτό δημιουργούμε ένα ευρετήριο λιγότερο και έτσι επιβαρύνουμε στο ελάχιστο λειτουργίες όπως η καταχώρηση και η ενημέρωση των αναρτήσεων.

2.

Εκτελούμε το Επερώτημα E3 και διαπιστώνουμε ότι εκτελείτε αισθητά πιο αργά σε σχέση με το E2. Εξετάζοντας το πλάνο εκτέλεσης του ερωτήματος (βλέπε επόμενη σελίδα) διαπιστώνουμε ότι υπάρχουν **33450** αναρτήσεις στις οποίες έχει αποδοθεί ο όρος **SQL.** Την ίδια στιγμή ο optimizer εκτιμά (βάσει των στατιστικών) ότι σε κάθε όρο αντιστοιχούν **129,5** αναρτήσεις. Στην περίπτωση του E2 ο όρος **Informix** είχε αποδοθεί σε **283** αναρτήσεις.



Λύση Τρίτου Ζητήματος

Δημιουργία Ευρετηρίου

```
create index idx_score on posts (score) include (owneruserid)
```

Q1

```
select  userid, displayname
     from users
  where not exists
        (select owneruserid  from posts
             where userid=owneruserid and score >= 0 and
                                      owneruserid is not null)
     and exists (select owneruserid  from posts
             where userid=owneruserid and score < 0 and
                                       owneruserid is not null)
```

Q2

```
select    distinct   userid, displayname
   from users, posts
 where userid = owneruserid and score  < 0  and userid not in
        (  select  owneruserid
                 from  posts where score >=0 and owneruserid is not null )
```

Q3

```
select userid, displayname
  from users, posts
 where users.userid = posts.owneruserid and score < 0

 except

 select userid, displayname
  from users, posts
 where users.userid = posts.owneruserid and score >= 0
```

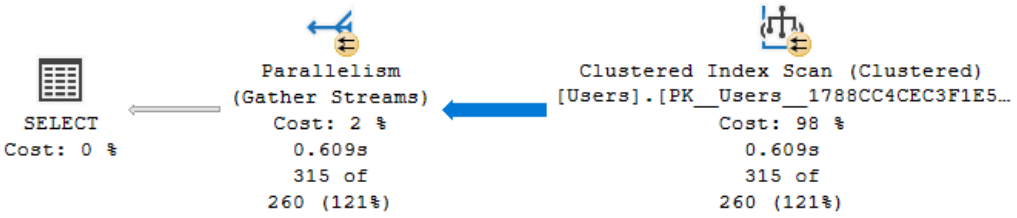
Τα επερωτήματα Q1 και Q2 είναι σχεδόν ισοδύναμα. Αν τα εκτελέσουμε και τα τρία σε δέσμη καταλαμβάνουν το 29%, 30% και 41% αντίστοιχα του συνολικού χρόνου εκτέλεσης.

Λύση Πέμπτου Ζητήματος

1.

Πλάνο εκτέλεσης του επερωτήματος δίχως ευρετήριο

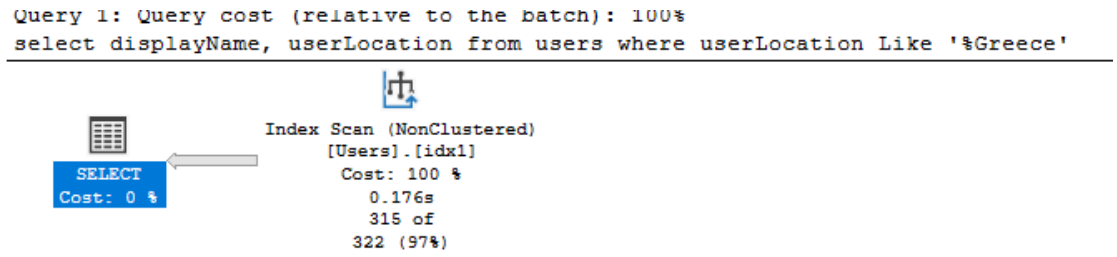
```
Query 1: Query cost (relative to the batch): 100%
select displayName, userLocation from users where userLocation Like '%Greece'
```



Το σύστημα διαβάζει ολόκληρο τον πίνακα.

Πλάνο εκτέλεσης μετά την δημιουργία του ευρετηρίου:

```
create index idx1 on users (userLocation) include (displayName)
```



Το σύστημα διαβάζει ολόκληρο το ευρετήριο (index scan) αφού σε αυτό μπορεί να βρει όλες τις πληροφορίες που χρειάζονται για να απαντηθεί το επερώτημα. Συνεπώς το ευρετήριο αξιοποιείται.

2.

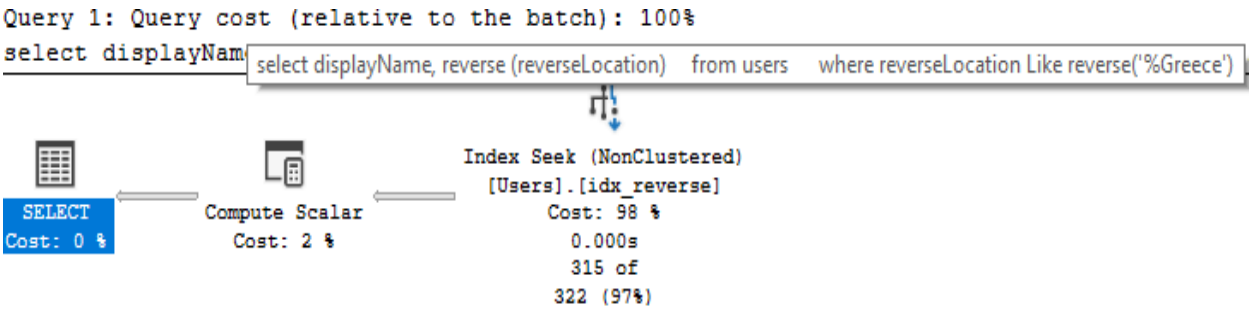
Προτεινόμενη Λύση Α

```
alter table users
    add  reversedLocation nvarchar(100)

update users
   set reversedLocation = reverse(userLocation)

create index idx_reverse on users (reversedLocation) include (displayName)

select displayName, reverse (reversedLocation)
   from users
    where reversedLocation Like reverse('%Greece')
```



Το σύστημα χρησιμοποιεί με τον πλέον αποδοτικό τρόπο το ευρετήριο (index seek).

Προτεινόμενη Λύση Α

Δημιουργία FULL TEXT INDEX στο πεδίο userLocation του πίνακα users.

```
CREATE FULLTEXT CATALOG ftc;

CREATE FULLTEXT INDEX ON users(userLocation)
   KEY INDEX   PK__Users__1788CC4C07D2DFB8 ON ftc;  --primary key of users

SELECT displayName FROM users WHERE contains(userlocation,'Greece')
```